

UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA



EJERCICIOS DE CLASE Nº 06

NOMBRE COMPLETO: Luis Daniel Salazar Islas

Nº de Cuenta: 320335163

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 06

SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 07/10/2025

CALIFICACIÓN: _____

EJERCICIOS DE SESIÓN:

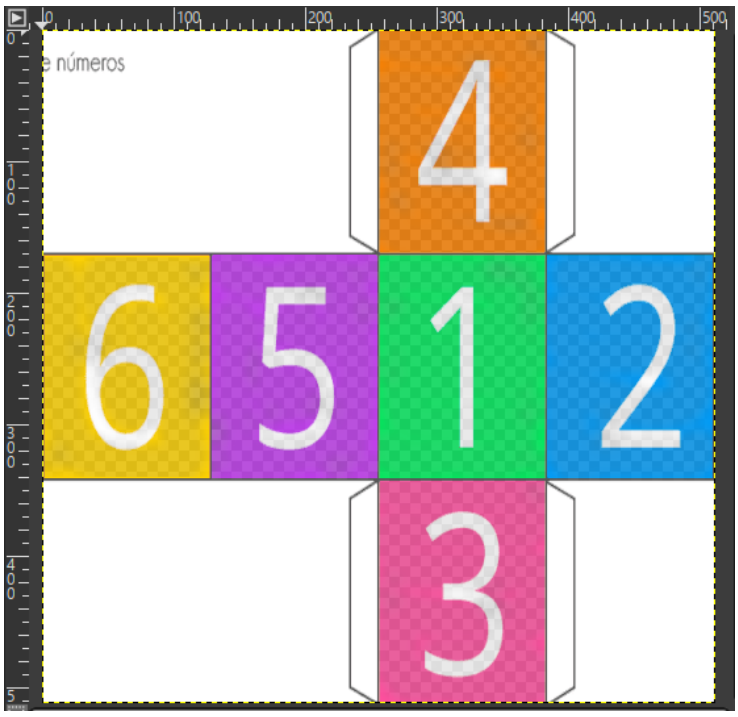
1. Actividades realizadas. Una descripción de los ejercicios y capturas de pantalla de bloques de código generados y de ejecución del programa

   Para esta práctica se deben diseñar dos dados a partir de texturas. En mi caso escogí los dados de números.

   Dado en GIMP

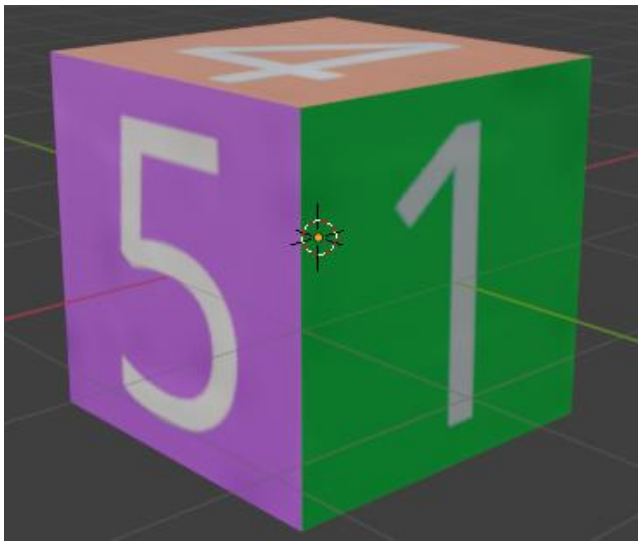
   El segundo dado se debe texturizar mediante OpenGL. Para ello primero necesitamos ajustar la imagen con los 3 requisitos:
   - Formato que soporte RGBA
   - Dimensiones de $2^n x 2^n$
   - Formato png, tga, etc.

Ello se realizó en GIMP mediante recorte, escalamiento y ajuste a formato RGB. Como extra añadí el canal alfa y ajusté la opacidad a 0.22 como en el laboratorio.



Dado en blender

El primer dado se debe diseñar con blender, a partir del cubo inicial y la imagen tga producida de gimp. Se sigue el procedimiento dado en laboratorio.



Se importó el modelo en obj y se mostró en blender

```
//Dado importado
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-4.0f, 4.5f, -2.0f));
model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 1.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
Dado_M.RenderModel();
```

Dado OpenGL

Importación de la imagen en tga

```
dadoTexture = Texture("Textures/dado-gimp.tga");
dadoTexture.LoadTextureA();
```

Se establecen las coordenadas de texturas, en algunos casos las coordenadas se deben establecer en un orden diferente, esto debido a que caras como la trasera o baja deben cambiar de orientación para armar el dado.

```
GLfloat cubo_vertices[] = {
    // front
    //x     y       z       S       T           NX      NY      NZ
    -0.5f, -0.5f,  0.5f,    0.26f,  0.34f,      0.0f,   0.0f,   -1.0f,  //0
    0.5f, -0.5f,  0.5f,     0.49f,  0.34f,      0.0f,   0.0f,   -1.0f,  //1
    0.5f,  0.5f,  0.5f,     0.49f,  0.66f,      0.0f,   0.0f,   -1.0f,  //2
    -0.5f,  0.5f,  0.5f,    0.26f,  0.66f,      0.0f,   0.0f,   -1.0f,  //3
    // right
    //x     y       z       S       T
    0.5f, -0.5f,  0.5f,     0.5f,  0.34f,       -1.0f,  0.0f,   0.0f,//4
    0.5f, -0.5f,  -0.5f,    0.74f,  0.34f,      -1.0f,  0.0f,   0.0f,//5
    0.5f,  0.5f,  -0.5f,    0.74f,  0.66f,      -1.0f,  0.0f,   0.0f,//6
    0.5f,  0.5f,  0.5f,     0.5f,   0.66f,      -1.0f,  0.0f,   0.0f,//7
    // back
    -0.5f, -0.5f, -0.5f,    0.99f,  0.34f,      0.0f,   0.0f,   1.0f,//8
    0.5f, -0.5f, -0.5f,     0.76f,  0.34f,      0.0f,   0.0f,   1.0f,//9
    0.5f,  0.5f, -0.5f,     0.76f,  0.65f,      0.0f,   0.0f,   1.0f,//10
    -0.5f,  0.5f, -0.5f,    0.99f,  0.65f,      0.0f,   0.0f,   1.0f,//11

    // left
    //x     y       z       S       T
    -0.5f, -0.5f,  -0.5f,   0.51f,  0.01f,      1.0f,   0.0f,   0.0f,//12
    -0.5f, -0.5f,  0.5f,    0.74f,  0.01f,      1.0f,   0.0f,   0.0f,//13
    -0.5f,  0.5f,  0.5f,    0.74f,  0.32f,      1.0f,   0.0f,   0.0f,//14
    -0.5f,  0.5f,  -0.5f,   0.51f,  0.32f,      1.0f,   0.0f,   0.0f,//15

    // bottom
    //x     y       z       S       T
    -0.5f, -0.5f,  0.5f,    0.01f,  0.65f,      0.0f,   1.0f,   0.0f,
    0.5f,  -0.5f,  0.5f,    0.24f,  0.65f,      0.0f,   1.0f,   0.0f,
     0.5f,  -0.5f,  -0.5f,  0.24f,  0.34f,      0.0f,   1.0f,   0.0f,
    -0.5f, -0.5f,  -0.5f,   0.01f,  0.34f,      0.0f,   1.0f,   0.0f,
```

```
    //UP
    //x       y       z       S       T
     -0.5f, 0.5f,  0.5f,    0.51f,  0.67f,      0.0f,   -1.0f,  0.0f,
     0.5f,  0.5f,  0.5f,    0.74f,  0.67f,      0.0f,   -1.0f,  0.0f,
      0.5f, 0.5f,  -0.5f,   0.74f,  0.99f,      0.0f,   -1.0f,  0.0f,
     -0.5f, 0.5f,  -0.5f,   0.51f,  0.99f,      0.0f,   -1.0f,  0.0f,

};
```

Renderización del cubo, se colocan unas instrucciones extra para permitir la propiedad alfa.

```
model = glm::mat4(1.0);
model = glm::translate(model, glm::vec3(-1.5f, 4.5f, -2.0f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
dadoTexture.UseTexture();
glEnable(GL_BLEND);
glBlendFunc(GL_SRC_ALPHA, GL_ONE_MINUS_SRC_ALPHA);
meshList[4]->RenderMesh();
```

Ejecución







2. Problemas presentados. Listar si surgieron problemas a la hora de ejecutar el código

   Para mí lo más difícil del ejercicio fue rotar las coordenadas de textura al momento de mapear las caras trasera y baja del dado, no lograba visualizar que cambios tenía que hacer. Así que me tomó tiempo entender de que forma se deben alternar el orden de las coordenadas para realizar las rotaciones adecuadas. Una vez entendido ese aspecto el resto fue sencillo.

3. Conclusión:
   a. Los ejercicios de la clase: Complejidad, explicación

   Los ejercicios me parecieron adecuados para el tema, por lo que he visto el tema de texturización es muy amplio, por lo que empezar con una figura simple como el cubo me ayudó a no perderme durante el proceso de texturización dentro del modelador y OpenGL. Siento que realizar los ejercicios fue una cuestión de repasar con los videos y tener paciencia para colocar bien las coordenadas de textura.

   b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias.

   En general la explicación del tema fue correcta, es extensa, pero entiendo que el tema lo es. Sinceramente fue de mucha ayuda que hubieran videos que consultar cada vez que me estancaba, por lo que en general siento que con el material proporcionado se podía realizar sin muchas dificultades la práctica.